

Secure Programming – Final Project Report

1. Introduction

As part of the Secure Programming course, our team set out to build a fully functional mini e-commerce web application from scratch. The goal was not just to make something that works, but to build it the right way — with security baked into every layer of the system from day one.

We chose to build **SecureShop**, a web-based online store where customers can browse products, add items to their cart, and place orders. The system also supports staff members who can manage and accept orders, and an administrator who has full control over products, users, and analytics.

Throughout the development process, we paid close attention to the OWASP Top 10 vulnerabilities and made deliberate design choices to protect against SQL injection, cross-site scripting (XSS), cross-site request forgery (CSRF), and other common attack vectors. This report documents our approach, the security measures we implemented, the threat modeling we performed, and the results of our security testing.

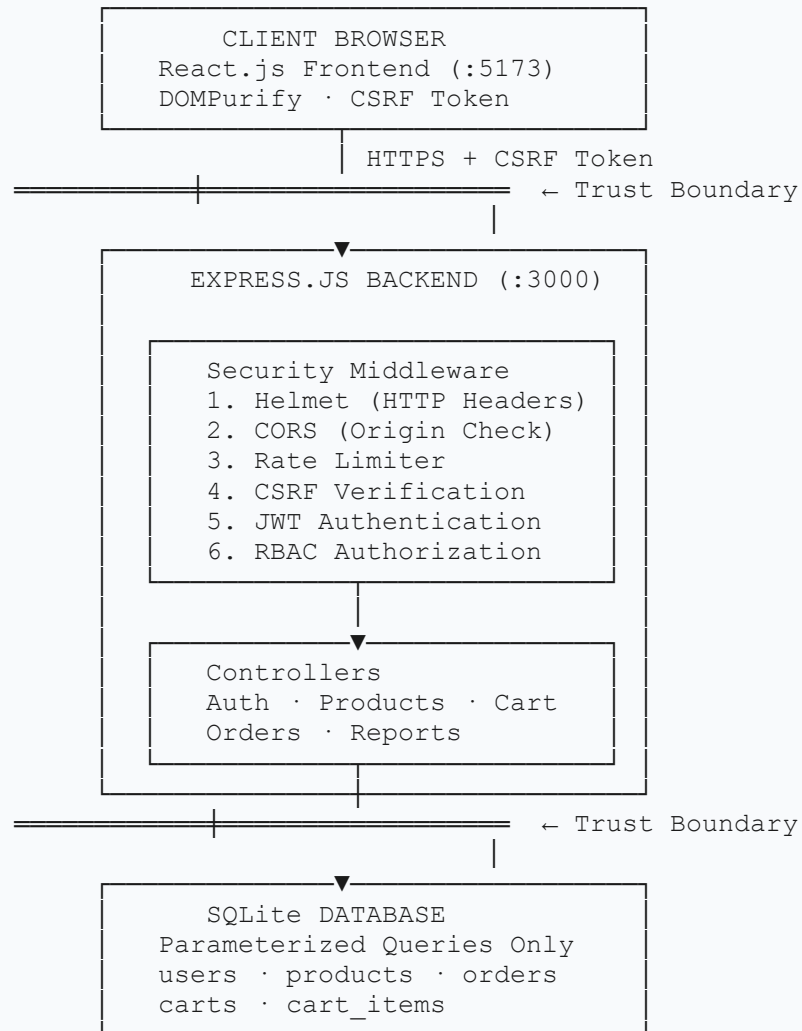
2. Technology Stack

We carefully selected our technology stack based on security considerations and ease of implementing secure patterns:

Layer	Technology	Why We Chose It
Frontend	React.js with Vite	Component-based architecture makes it easier to isolate and sanitize user inputs at each UI level
Backend	Node.js + Express.js	Rich middleware ecosystem for security (helmet, cors, csrf, rate-limit)
Database	SQLite	Lightweight relational DB that still supports parameterized queries, perfect for demonstrating SQL injection prevention
Authentication	JWT in HttpOnly Cookies	Prevents token theft via XSS since JavaScript cannot access HttpOnly cookies
Password Security	bcrypt (10 rounds)	Industry-standard adaptive hashing that is resistant to rainbow table and brute-force attacks
Input Validation	express-validator	Server-side validation library that sanitizes and validates all user inputs before they reach the database

3. System Architecture

Our application follows a classic three-tier architecture with a clear separation between the presentation layer, business logic, and data storage. What makes our design security-focused is the multiple layers of middleware that every request must pass through before reaching any business logic.



4. User Roles and Access Control

We implemented role-based access control (RBAC) with three distinct roles. One important security decision we made early on was to **never allow admin or staff registration through the application interface**. These privileged accounts are created only through a secure server-side seeding script that runs at startup.

Role	Login Credentials	What They Can Do
Admin	admin@seureshop.com / Admin@123	Full system access: manage products (CRUD), view all orders, change order status, access reports and analytics, view customer data
Staff	yusufwardany / hi123456	Order management: view all customer orders, accept pending orders, update order status (Processing → Shipped → Delivered), view sales reports
Customer	Register through the app	Shopping: browse and search products, add to cart, place orders with multi-step checkout, view own order history with progress tracking

Security Note: The registration page only creates Customer accounts. There is no option to select a role during registration. Even if someone tries to send a role field in the API request body, the server ignores it and forces the role to "Customer". Admin and Staff accounts can only be created by modifying the seed script and restarting the server.

5. Security Features Implementation

5.1 Input Validation (Server-Side)

We used the `express-validator` library to validate and sanitize all user inputs on the server side. We want to emphasize that client-side validation alone is never sufficient — it can be easily bypassed by directly sending requests to the API. That is why every validation rule is enforced on the server.

```
// Server-side validation rules for user registration

const registerValidation = [

  body('username').trim().isLength({ min: 3, max: 50 })

    .withMessage('Username must be 3-50 characters').escape(),

  body('email').isEmail().normalizeEmail(),

  body('password').isLength({ min: 6 })

    .matches(/[A-Z]/).withMessage('Must contain uppercase')

    .matches(/[0-9]/).withMessage('Must contain a number')

];
```

5.2 SQL Injection Prevention

Every single database query in our application uses parameterized statements. We never concatenate user input into SQL strings. The database wrapper we built accepts parameters as separate arguments, and the SQLite driver handles proper escaping automatically.

```
// SECURE: The $1 placeholder ensures user input is treated as data, not SQL code

const result = await db.query(

  'SELECT id, password_hash, role FROM users WHERE email = $1',

  [email] // This value can never break out of the string context

);

// VULNERABLE (we never do this):

// const query = "SELECT * FROM users WHERE email = '" + email + "'";
```

5.3 Password Hashing with bcrypt

We never store passwords in plain text. Every password goes through bcrypt hashing with a cost factor of 10 before being saved to the database. bcrypt is specifically designed for password hashing — it is intentionally slow, which makes brute-force attacks impractical. It also automatically generates a unique salt for each password, so even if two users have the same password, their hashes will be different.

```
// During registration – hash before storing

const passwordHash = await bcrypt.hash(password, 10);

// During login – timing-attack resistant comparison

const isMatch = await bcrypt.compare(password, user.password_hash);
```

5.4 XSS Protection

We tackle cross-site scripting from both sides. On the server, the Helmet middleware sets security headers including X-XSS-Protection. On the client, we use the DOMPurify library to sanitize any HTML content before rendering it. React also escapes content by default when using JSX expressions.

```
// Client-side: DOMPurify strips malicious HTML/JS before rendering

<div dangerouslySetInnerHTML={{
  __html: DOMPurify.sanitize(product.description)
}} />
```

5.5 CSRF Protection

Cross-site request forgery is prevented using the csrf middleware. When the frontend loads, it requests a CSRF token from the server. This token is then attached to every state-changing request (POST, PUT, DELETE). If the token is missing or invalid, the server rejects the request with a 403 error.

5.6 Secure Session Management

Instead of storing JWT tokens in localStorage (which is vulnerable to XSS attacks), we store them in HttpOnly cookies. This means JavaScript running in the browser cannot access the token at all. We also set the SameSite attribute to "lax" to provide an additional layer of CSRF protection.

```
// JWT stored in HttpOnly cookie – invisible to JavaScript

res.cookie('jwt', token, {
```

```
httpOnly: true, // JavaScript cannot access this cookie

sameSite: 'lax', // Not sent with cross-site requests

secure: process.env.NODE_ENV === 'production',

maxAge: 7200000 // Expires after 2 hours

});
```

5.7 Account Lockout

To prevent brute-force password guessing attacks, we implemented an account lockout mechanism. After 5 consecutive failed login attempts, the account is locked for 15 minutes. This works alongside the rate limiter, which restricts auth endpoints to 30 requests per 15-minute window.

5.8 Secure Error Handling

A common mistake in web applications is leaking internal error details (stack traces, SQL error messages, file paths) to the user. Our global error handler catches all errors and returns a generic message to the client while logging the actual error details on the server for debugging.

5.9 HTTP Security Headers (Helmet)

Helmet automatically sets several HTTP headers that protect against common attacks: X-Content-Type-Options prevents MIME sniffing, X-Frame-Options prevents clickjacking, and Strict-Transport-Security enforces HTTPS connections.

5.10 Logging and Monitoring

We use Morgan middleware to log every HTTP request, including the IP address, method, URL, status code, and response time. Security-relevant events like failed logins and account lockouts are logged separately with descriptive messages.

6. Threat Modeling – STRIDE Analysis

We used the STRIDE methodology to systematically identify potential threats to our system. For each threat category, we identified the specific risk, assessed its impact, and documented how we mitigated it.

Category	Threat Description	Our Mitigation	Status
S – Spoofing	Attacker creates an admin account via the registration form	Registration endpoint forces role to "Customer". Admin/Staff are only seeded server-side.	✓ Mitigated
S – Spoofing	Attacker steals JWT token through XSS	JWT stored in HttpOnly cookie (not accessible via JavaScript). SameSite attribute set.	✓ Mitigated
T – Tampering	SQL injection to manipulate database	All queries use parameterized statements (\$1, \$2 placeholders). No string concatenation.	✓ Mitigated
T – Tampering	XSS attack to inject malicious scripts	DOMPurify sanitization, Helmet XSS headers, React default escaping, express-validator escape().	✓ Mitigated
T – Tampering	CSRF attack to perform unauthorized actions	csrf middleware validates token on all POST/PUT/DELETE requests.	✓ Mitigated
R – Repudiation	User denies performing an action	Morgan request logging with timestamps, IP addresses, and response codes.	✓ Mitigated
I – Info Disclosure	Passwords exposed from database breach	bcrypt hashing with unique salts. Original passwords cannot be recovered.	✓ Mitigated

I – Info Disclosure	Stack traces reveal system internals	Global error handler returns generic messages only. Details logged server-side.	✓ Mitigated
D – Denial of Service	Brute-force login flooding	Rate limiting (30 req/15min on auth), account lockout after 5 failed attempts.	✓ Mitigated
E – Elevation of Privilege	Customer accesses admin functionality	RBAC middleware (requireAdmin, requireStaff) on all protected routes.	✓ Mitigated

6.1 Risk Assessment Matrix

Risk	Likelihood (1-5)	Impact (1-5)	Score	Priority
SQL Injection	1	5	5	Low (mitigated)
XSS Attack	1	4	4	Low (mitigated)
CSRF Attack	1	4	4	Low (mitigated)
Brute Force Login	2	3	6	Medium (mitigated)
Privilege Escalation	1	5	5	Low (mitigated)
Password Theft	1	5	5	Low (mitigated)
Session Hijacking	1	4	4	Low (mitigated)

7. Security Testing Results

We conducted several manual penetration tests to verify that our security controls actually work as intended. Below are the tests we performed and their outcomes.

7.1 SQL Injection Test

What we tried: We entered the classic SQL injection payload `' OR 1=1 --` into the email field on the login form and submitted the request.

What happened: The login failed with the message "Invalid email or password." The parameterized query treated the entire input as a literal string value, not as SQL code. The database searched for a user with the email literally equal to `' OR 1=1 --`, found nothing, and returned an empty result.

Conclusion: SQL injection is effectively blocked by our parameterized query approach.

7.2 XSS Test

What we tried: We attempted to inject a script tag `<script>alert('XSS')</script>` into a product description field through the admin panel.

What happened: The script tag was stripped out by DOMPurify before being rendered on the page. The product description displayed as plain text without executing any JavaScript code.

Conclusion: Both stored and reflected XSS attacks are prevented by our multi-layered approach (Helmet headers + DOMPurify + React escaping).

7.3 CSRF Test

What we tried: We used a tool to send a POST request to `/api/auth/register` without including the CSRF token in the headers.

What happened: The server responded with a 403 Forbidden error and the message "Invalid CSRF Token. Please refresh the page."

Conclusion: State-changing requests cannot be forged from external sites because they require a valid CSRF token that only our frontend can obtain.

7.4 Authentication Bypass Test

What we tried: We sent a GET request to the admin-only endpoint `/api/orders/all` without any authentication cookie, and then again with a Customer-role JWT.

What happened: Without a cookie, we got a 401 "Authentication required" error. With a Customer JWT, we got a 403 "Staff access required" error. Only Staff and Admin JWTs are accepted.

Conclusion: Both authentication and authorization are properly enforced at the middleware level.

7.5 Privilege Escalation Test

What we tried: We sent a POST request to `/api/auth/register` with the body `{"role": "Admin", "username": "hacker", ...}`.

What happened: The account was created successfully, but with the role "Customer" — the server completely ignored the role field we sent. The backend code hardcodes the role to "Customer" for all registrations.

Conclusion: It is impossible to create privileged accounts through the registration endpoint.

8. Application Features Summary

Feature	Description
Product Catalog	32 products across 6 categories with search, filtering, sorting, and pagination
Shopping Cart	Add/remove items, quantity controls, stock validation
Multi-Step Checkout	Cart → Shipping Info → Order Confirmation → Receipt
Order Auto-Accept	Orders are automatically set to "Processing" status upon placement
Order Tracking	Visual progress bar: Ordered → Processing → Shipped → Delivered
Staff Order Management	Staff can accept, process, ship, and deliver orders
Admin Dashboard	Product CRUD with image upload, order management, 6 stat cards
Reports & Analytics	Donut charts, bar charts, data tables for sales, revenue, stock, customers
Dark/Light Mode	Theme toggle with localStorage persistence
Toast Notifications	Success/error/info notifications with auto-dismiss animation
Password Strength Meter	Real-time visual feedback during registration (weak/medium/strong)

Responsive Design

Works on desktop and mobile with CSS Grid and media queries

9. Security Features Checklist

#	Security Feature	Implementation	File(s)
1	Server-side input validation	express-validator on all routes	routes/*.js
2	Parameterized SQL queries	\$1, \$2 placeholders via db.query()	controllers/*.js
3	Password hashing	bcrypt with 10 salt rounds	authController.js
4	XSS prevention (server)	Helmet security headers	app.js
5	XSS prevention (client)	DOMPurify sanitization	Dashboard.jsx
6	CSRF protection	csrf token exchange	app.js, api.js
7	Secure session (JWT)	HttpOnly + SameSite cookies	authController.js
8	RBAC (3 roles)	requireAuth/Admin/Staff middleware	middleware/auth.js
9	Rate limiting	30 req/15min on auth routes	app.js
10	Account lockout	5 failed attempts → 15 min lock	authController.js

11	Secure error handling	Generic errors, no stack traces	app.js
12	HTTP request logging	Morgan middleware	app.js
13	CORS restriction	Only frontend origin allowed	app.js
14	No admin registration	Privileged accounts seeded only	seedAdmin.js
15	File upload validation	2MB limit, image types only	productRoutes.js

10. How to Run the Application

```
# Step 1: Install backend dependencies

cd backend
npm install

# Step 2: Start the backend server (auto-seeds admin, staff, and products)

node server.js

# Step 3: In a new terminal, install frontend dependencies

cd frontend
npm install

# Step 4: Start the frontend development server

npm run dev

# Step 5: Open your browser and go to

http://localhost:5173
```

11. Conclusion

Building SecureShop taught us that security is not something you bolt on at the end — it needs to be part of every design decision from the very beginning. By following the principle of defense in depth, we created multiple layers of protection so that even if one layer fails, the others still protect the system.

We addressed every category of the OWASP Top 10, from broken access control to injection attacks. Our STRIDE analysis helped us think like attackers and proactively identify threats before they became vulnerabilities. And our penetration testing confirmed that our security measures actually work as intended.

The most valuable lesson we learned is that secure code does not have to be complicated. Simple practices like using parameterized queries, hashing passwords with bcrypt, and storing tokens in HttpOnly cookies go a long way toward building a system that is resilient against the most common web attacks.

12. Team members

Youssif Yasser wardany

22100809

Hazem khaled kassem

22101258

Omar islam